Tarea 5

Strassen vs. Fuerza Bruta: Multiplicación de Matrices

Autor: Víctor Manuel García Castañeda

Curso: Diseño de Algoritmos

Fecha: Abril 2026

Universidad Autónoma de Manizales

Departamento de Ingeniería de Sistemas

Índice

1. Introducción **2**

2. Descripción de los Algoritmos **2**
2.1. Algoritmo de Fuerza Bruta — $O(n^3)$ 2
2.2. Algoritmo de Strassen — $O(n^{\log_2 7})$ 3
2.3. Variante Híbrida 3

3. Análisis Teórico **3**
3.1. Fuerza Bruta 3
3.2. Strassen — Teorema Maestro 3
3.3. Comparación Asintótica 4

4. Implementación **4**
4.1. Strassen Híbrido (`strassen.cpp`) 4
4.2. Fuerza Bruta (`brute_force.cpp`) 7
4.3. Generador de Instancias (`generator.py`) 8
4.4. Script de Gráficas (`graficar.py`) 10

5. Resultados Experimentales **12**
5.1. Metodología 12
5.2. Fase 1 — Strassen Puro (sin umbral híbrido) 12
5.3. Fase 2 — Strassen Híbrido (umbral $n_0 = 32$) 12

6. Análisis Gráfico **13**

7. Conclusiones **16**

1. Introducción

La multiplicación de matrices es una de las operaciones fundamentales en las ciencias de la computación y las matemáticas aplicadas. Su importancia práctica abarca desde la resolución de sistemas de ecuaciones lineales y el cálculo de transformaciones geométricas, hasta el entrenamiento de redes neuronales profundas y la simulación física [1].

El algoritmo clásico para multiplicar dos matrices cuadradas de tamaño $n \times n$ requiere $\Theta(n^3)$ operaciones aritméticas: por cada uno de los n^2 elementos del resultado se realiza un producto interno de longitud n. Aunque conceptualmente sencillo, su crecimiento cúbico lo hace impracticable para matrices de gran tamaño [6].

En 1969, Volker Strassen demostró que es posible multiplicar matrices 2×2 usando únicamente **7 multiplicaciones** en lugar de las 8 del método clásico, sacrificando algunas sumas adicionales [2]. Aplicado de forma recursiva mediante el paradigma *Dividir y Conquistar*, este resultado reduce la complejidad teórica a $O(n^{\log_2 7}) \approx O(n^{2.807})$, una mejora asintótica significativa frente a $O(n^3)$ [1].

El presente informe documenta la implementación, instrumentación, evaluación experimental y análisis teórico de ambos algoritmos. En particular, se estudia el punto de cruce (*crossover point*) a partir del cual la variante híbrida de Strassen supera en tiempo real a la fuerza bruta, y se contrastan los resultados empíricos con las cotas teóricas [7].

2. Descripción de los Algoritmos

2.1. Algoritmo de Fuerza Bruta — $O(n^3)$

La multiplicación clásica de dos matrices $A, B \in \mathbb{R}^{n\times n}$ produce $C \in \mathbb{R}^{n\times n}$ según:

$$C_{ij} = \sum_{k=1}^{n} A_{ik} \cdot B_{kj}, \quad 1 \le i, j \le n. \tag{1}$$

Ejemplo numérico ($n = 2$). Sean

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

Aplicando la ecuación (1) elemento a elemento:

$$\begin{aligned} C_{11} &= 1{\cdot}5 + 2{\cdot}7 = 19, \quad & C_{12} &= 1{\cdot}6 + 2{\cdot}8 = 22, \\ C_{21} &= 3{\cdot}5 + 4{\cdot}7 = 43, \quad & C_{22} &= 3{\cdot}6 + 4{\cdot}8 = 50. \end{aligned}$$

Por tanto $C = \left(\begin{smallmatrix} 19 & 22 \\ 43 & 50 \end{smallmatrix}\right)$.

La implementación directa con tres bucles anidados ejecuta exactamente n^3 multiplicaciones y $n^3 - n^2$ sumas, para un total de $\Theta(n^3)$ operaciones elementales [1].

2.2. Algoritmo de Strassen — $O(n^{\log_2 7})$

El algoritmo de Strassen divide cada matriz en cuatro submatrices de tamaño $\frac{n}{2} \times \frac{n}{2}$ [2]:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}.$$

En lugar de 8 multiplicaciones de submatrices, define **7 productos auxiliares**:

$$\begin{aligned}
M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}), \\
M_2 &= (A_{21} + A_{22})\, B_{11}, \\
M_3 &= A_{11}(B_{12} - B_{22}), \\
M_4 &= A_{22}(B_{21} - B_{11}), \\
M_5 &= (A_{11} + A_{12})\, B_{22}, \\
M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}), \\
M_7 &= (A_{12} - A_{22})(B_{21} + B_{22}).
\end{aligned}$$

Los cuadrantes del resultado se recuperan mediante sumas y restas:

$$C_{11} = M_1 + M_4 - M_5 + M_7, \quad C_{12} = M_3 + M_5, \quad C_{21} = M_2 + M_4, \quad C_{22} = M_1 - M_2 + M_3 + M_6.$$

2.3. Variante Híbrida

En la práctica, la constante oculta en $\Theta(n^{2.807})$ es grande: la recursion genera submatrices temporales, 18 sumas/restas adicionales y overhead de llamadas de función [3]. La implementación estándar en la industria es un **Strassen híbrido**: se aplica la recursion hasta que las submatrices alcanzan un umbral n_0, a partir del cual se usa la fuerza bruta [4]. En esta implementación se estableció $n_0 = 32$, determinado empíricamente. La complejidad asintótica no cambia: sigue siendo $O(n^{\log_2 7})$ [1].

3. Análisis Teórico

3.1. Fuerza Bruta

El costo del algoritmo clásico es $T_{\text{bruta}}(n) = \Theta(n^3)$. La función de menor crecimiento tal que el algoritmo sea $O(f(n))$ es $\boxed{f(n) = n^3}$.

3.2. Strassen — Teorema Maestro

Aplicando Strassen de forma recursiva, el número de operaciones satisface:

$$T(n) = 7\,T\left(\tfrac{n}{2}\right) + \Theta(n^2). \tag{2}$$

Se aplica el **Teorema Maestro** [1] con $a = 7$, $b = 2$ y $f(n) = \Theta(n^2)$:

$$n^{\log_b a} = n^{\log_2 7} \approx n^{2.807} \succ n^2 = f(n).$$

Como $f(n) = O\big(n^{\log_2 7 - \varepsilon}\big)$ con $\varepsilon = \log_2 7 - 2 \approx 0.807 > 0$, aplica el **Caso 1**:

$$\boxed{T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})}. \tag{3}$$

3.3. Comparación Asintótica

La razón de instrucciones esperada entre fuerza bruta y Strassen crece como:

$$\frac{T_{\text{bruta}}(n)}{T_{\text{Strassen}}(n)} \approx \frac{n^3}{n^{2.807}} = n^{0.193} \xrightarrow{n\to\infty} \infty.$$

Para $n = 1024$: $1024^{0,193} \approx 3.0$, predicción confirmada experimentalmente en la Sección 5.

4. Implementación

Todos los programas se compilaron con `g++ -O0` para deshabilitar optimizaciones del compilador y medir el comportamiento real del algoritmo. Los archivos fuente (`strassen.cpp`, `brute_force.cpp`, `generator.py` y `graficar.py`), junto con las instancias generadas y los resultados en CSV, se encuentran disponibles en el archivo comprimido `proyecto_matrices.zip` adjunto a esta entrega.

Formato del archivo de entrada (`.txt`):

```
n
a_11  a_12  ...  a_1n
...
b_11  b_12  ...  b_1n
...
```

Formato del archivo de salida:

```
tiempo: X.XXXXXX segundos
instrucciones: XXXXXXXX
c_11  c_12  ...  c_1n
...
```

4.1. Strassen Híbrido (`strassen.cpp`)

La implementación usa `THRESHOLD = 32`. Cuando el tamaño de la submatriz es ≤ 32, se invoca la función de fuerza bruta. Para matrices cuyo tamaño no es potencia de dos, se aplica *padding* de ceros hasta la potencia de dos inmediatamente superior, y el resultado se recorta al tamaño original.

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>
#include <cmath>
```

```
#include <string>
using namespace std;
typedef vector<vector<long long>> Matrix;

long long instruction_count = 0;
const int THRESHOLD = 32;

Matrix add(const Matrix& A, const Matrix& B) {
    int n = A.size();
    Matrix C(n, vector<long long>(n, 0));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            instruction_count++;
            C[i][j] = A[i][j] + B[i][j];
        }
    return C;
}

Matrix sub(const Matrix& A, const Matrix& B) {
    int n = A.size();
    Matrix C(n, vector<long long>(n, 0));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            instruction_count++;
            C[i][j] = A[i][j] - B[i][j];
        }
    return C;
}

Matrix bruteForce(const Matrix& A, const Matrix& B) {
    int n = A.size();
    Matrix C(n, vector<long long>(n, 0));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            for (int k = 0; k < n; k++) {
                instruction_count++;
                C[i][j] += A[i][k] * B[k][j];
            }
    return C;
}

Matrix strassen(const Matrix& A, const Matrix& B) {
    int n = A.size();
    instruction_count++;
    if (n <= THRESHOLD) {
        instruction_count++;
        return bruteForce(A, B);
    }
    int half = n / 2;
    Matrix A11(half, vector<long long>(half));
    Matrix A12(half, vector<long long>(half));
    Matrix A21(half, vector<long long>(half));
    Matrix A22(half, vector<long long>(half));
    Matrix B11(half, vector<long long>(half));
    Matrix B12(half, vector<long long>(half));
    Matrix B21(half, vector<long long>(half));
    Matrix B22(half, vector<long long>(half));
    for (int i = 0; i < half; i++)
```

```
        for (int j = 0; j < half; j++) {
            instruction_count += 4;
            A11[i][j] = A[i][j];
            A12[i][j] = A[i][j + half];
            A21[i][j] = A[i + half][j];
            A22[i][j] = A[i + half][j + half];
            B11[i][j] = B[i][j];
            B12[i][j] = B[i][j + half];
            B21[i][j] = B[i + half][j];
            B22[i][j] = B[i + half][j + half];
            instruction_count += 4;
        }
    Matrix M1 = strassen(add(A11, A22), add(B11, B22));
    Matrix M2 = strassen(add(A21, A22), B11);
    Matrix M3 = strassen(A11, sub(B12, B22));
    Matrix M4 = strassen(A22, sub(B21, B11));
    Matrix M5 = strassen(add(A11, A12), B22);
    Matrix M6 = strassen(sub(A21, A11), add(B11, B12));
    Matrix M7 = strassen(sub(A12, A22), add(B21, B22));
    Matrix C11 = add(sub(add(M1, M4), M5), M7);
    Matrix C12 = add(M3, M5);
    Matrix C21 = add(M2, M4);
    Matrix C22 = add(sub(add(M1, M3), M2), M6);
    Matrix C(n, vector<long long>(n, 0));
    for (int i = 0; i < half; i++)
        for (int j = 0; j < half; j++) {
            instruction_count += 4;
            C[i][j]                 = C11[i][j];
            C[i][j + half]          = C12[i][j];
            C[i + half][j]          = C21[i][j];
            C[i + half][j + half]   = C22[i][j];
        }
    return C;
}

Matrix pad(const Matrix& A, int newSize) {
    Matrix P(newSize, vector<long long>(newSize, 0));
    for (int i = 0; i < (int)A.size(); i++)
        for (int j = 0; j < (int)A[0].size(); j++)
            P[i][j] = A[i][j];
    return P;
}

int main(int argc, char* argv[]) {
    if (argc != 3) {
        cerr << "Uso: " << argv[0] << " <entrada.txt> <salida.txt>" << endl;
        return 1;
    }
    ifstream fin(argv[1]);
    if (!fin) {
        cerr << "No se pudo abrir: " << argv[1] << endl;
        return 1;
    }
    int n;
    fin >> n;
    Matrix A(n, vector<long long>(n));
    Matrix B(n, vector<long long>(n));
```

```
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            fin >> A[i][j];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            fin >> B[i][j];
    fin.close();
    int newSize = 1;
    while (newSize < n) newSize *= 2;
    Matrix Apad = pad(A, newSize);
    Matrix Bpad = pad(B, newSize);
    auto start = chrono::high_resolution_clock::now();
    Matrix Cpad = strassen(Apad, Bpad);
    auto end = chrono::high_resolution_clock::now();
    double elapsed = chrono::duration<double>(end - start).count();
    Matrix C(n, vector<long long>(n));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            C[i][j] = Cpad[i][j];
    ofstream fout(argv[2]);
    fout << "tiempo: " << elapsed << " segundos" << endl;
    fout << "instrucciones: " << instruction_count << endl;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            fout << C[i][j];
            if (j < n - 1) fout << " ";
        }
        fout << endl;
    }
    fout.close();
    cout << "Strassen hibrido listo. Tiempo: " << elapsed
         << "s | Instrucciones: " << instruction_count << endl;
    return 0;
}
```

Listing 1: Código completo de `strassen.cpp`

4.2. Fuerza Bruta (`brute_force.cpp`)

Implementa directamente la ecuación (1) con tres bucles anidados. Cada instrucción simple se contabiliza explícitamente mediante el contador `instruction_count`.

```
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>
#include <string>
using namespace std;
typedef vector<vector<long long>> Matrix;

int main(int argc, char* argv[]) {
    if (argc != 3) {
        cerr << "Uso: " << argv[0] << " <entrada.txt> <salida.txt>" <<
            endl;
        return 1;
    }
    ifstream fin(argv[1]);
```

```
    if (!fin) {
        cerr << "No␣se␣pudo␣abrir:␣" << argv[1] << endl;
        return 1;
    }
    int n;
    fin >> n;
    Matrix A(n, vector<long long>(n));
    Matrix B(n, vector<long long>(n));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            fin >> A[i][j];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            fin >> B[i][j];
    fin.close();
    long long instruction_count = 0;
    Matrix C(n, vector<long long>(n, 0));
    auto start = chrono::high_resolution_clock::now();
    for (int i = 0; i < n; i++) {
        instruction_count++;
        for (int j = 0; j < n; j++) {
            instruction_count++;
            C[i][j] = 0;
            instruction_count++;
            for (int k = 0; k < n; k++) {
                instruction_count++;
                C[i][j] += A[i][k] * B[k][j];
                instruction_count++;
            }
        }
    }
    auto end = chrono::high_resolution_clock::now();
    double elapsed = chrono::duration<double>(end - start).count();
    ofstream fout(argv[2]);
    fout << "tiempo:␣" << elapsed << "␣segundos" << endl;
    fout << "instrucciones:␣" << instruction_count << endl;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            fout << C[i][j];
            if (j < n - 1) fout << "␣";
        }
        fout << endl;
    }
    fout.close();
    cout << "Fuerza␣Bruta␣lista.␣Tiempo:␣" << elapsed
         << "s␣|␣Instrucciones:␣" << instruction_count << endl;
    return 0;
}
```

Listing 2: Código completo de `brute_force.cpp`

4.3. Generador de Instancias (`generator.py`)

El generador en Python realiza de forma automática:

1. Genera matrices A y B de tamaño $n \times n$ con entradas aleatorias uniformes en $[-10, 10] \cap \mathbb{Z}$.

2. Escribe cada instancia en un archivo `.txt`.
3. Invoca los ejecutables `strassen` y `brute_force` vía `subprocess.run`.
4. Parsea los archivos de salida y construye un `DataFrame`.
5. Exporta `resultados.csv` y `resumen.csv`.

```python
import numpy as np
import subprocess
import os
import pandas as pd
import re

SIZES        = [2, 4, 8, 16, 32, 64, 128, 256, 512, 1024]
REPS         = 3
VAL_MIN      = -10
VAL_MAX      = 10
STRASSEN_BIN = "./strassen"
BRUTE_BIN    = "./brute_force"
INPUT_DIR    = "instances"
OUTPUT_DIR   = "outputs"

os.makedirs(INPUT_DIR,  exist_ok=True)
os.makedirs(OUTPUT_DIR, exist_ok=True)

def generate_instance(n, rep):
    A = np.random.randint(VAL_MIN, VAL_MAX + 1, size=(n, n))
    B = np.random.randint(VAL_MIN, VAL_MAX + 1, size=(n, n))
    fname = os.path.join(INPUT_DIR, f"matrix_n{n}_r{rep}.txt")
    with open(fname, "w") as f:
        f.write(f"{n}\n")
        for row in A:
            f.write(" ".join(map(str, row)) + "\n")
        for row in B:
            f.write(" ".join(map(str, row)) + "\n")
    return fname

def parse_output(filepath):
    with open(filepath) as f:
        lines = f.readlines()
    tiempo = float(re.search(
        r"tiempo:\s*([\d.e+\-]+)", lines[0]).group(1))
    instrucciones = int(re.search(
        r"instrucciones:\s*(\d+)", lines[1]).group(1))
    return tiempo, instrucciones

def run_program(binary, input_file, output_file):
    result = subprocess.run(
        [binary, input_file, output_file],
        capture_output=True, text=True)
    if result.returncode != 0:
        print(f"ERROR: {result.stderr}")
        return None, None
    return parse_output(output_file)

records = []
for n in SIZES:
    for rep in range(1, REPS + 1):
```

```
        print(f"  n={n:4d}  rep={rep}/{REPS}", end="  ...  ", flush=True)
        input_file   = generate_instance(n, rep)
        out_strassen = os.path.join(
            OUTPUT_DIR, f"out_strassen_n{n}_r{rep}.txt")
        out_brute    = os.path.join(
            OUTPUT_DIR, f"out_brute_n{n}_r{rep}.txt")
        t_str, i_str = run_program(STRASSEN_BIN, input_file,
           out_strassen)
        t_bru, i_bru = run_program(BRUTE_BIN,    input_file, out_brute)
        records.append({"n": n, "rep": rep,
                        "tiempo_strassen":        t_str,
                        "instrucciones_strassen": i_str,
                        "tiempo_bruta":           t_bru,
                        "instrucciones_bruta":    i_bru})
        print(f"Strassen: {t_str:.6f}s/{i_str} instr "
              f"| Bruta: {t_bru:.6f}s/{i_bru} instr")

df = pd.DataFrame(records)
summary = (df.groupby("n")
             .agg(tiempo_strassen_avg=("tiempo_strassen","mean"),
                  instrucciones_strassen_avg=("instrucciones_strassen","
                     mean"),
                  tiempo_bruta_avg=("tiempo_bruta","mean"),
                  instrucciones_bruta_avg=("instrucciones_bruta","mean")
                     )
             .reset_index())
df.to_csv("resultados.csv", index=False)
summary.to_csv("resumen.csv", index=False)
print("\n=== Resumen por tamano ===")
print(summary.to_string(index=False))
```

Listing 3: Código completo de `generator.py`

4.4. Script de Gráficas (`graficar.py`)

El script lee `resumen.csv`, calcula las constantes C_1 y C_2, filtra los tiempos iguales a cero y genera las tres gráficas.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

df     = pd.read_csv("resumen.csv")
n      = df["n"].values
t_str  = df["tiempo_strassen_avg"].values
t_bru  = df["tiempo_bruta_avg"].values
i_str  = df["instrucciones_strassen_avg"].values
i_bru  = df["instrucciones_bruta_avg"].values

exp_strassen = np.log2(7)
f_bruta      = n**3
f_strassen   = n**exp_strassen

mask_i = n >= 128
C_bruta_instr    = np.mean(i_bru[mask_i] / f_bruta[mask_i])
C_strassen_instr = np.mean(i_str[mask_i] / f_strassen[mask_i])
```

```

mask_t = (t_str > 0) & (t_bru > 0)
n_t = n[mask_t]; ts_t = t_str[mask_t]; tb_t = t_bru[mask_t]
C_bruta_time    = np.mean(tb_t / n_t**3)
C_strassen_time = np.mean(ts_t / n_t**exp_strassen)

fig, ax = plt.subplots(figsize=(9, 6))
ax.plot(n, i_bru, 'o-', color='#e74c3c', lw=2, ms=7,
        label='Fuerza Bruta (experimental)')
ax.plot(n, i_str, 's-', color='#2980b9', lw=2, ms=7,
        label='Strassen Hibrido (experimental)')
ax.plot(n, C_bruta_instr * f_bruta, '--', color='#c0392b', lw=1.5,
        label=f'C1*n^3 (C1={C_bruta_instr:.4f})')
ax.plot(n, C_strassen_instr * f_strassen, '--', color='#1a5276', lw=1.5,
        label=f'C2*n^log2(7) (C2={C_strassen_instr:.4f})')
ax.set_xlabel('Tamano n', fontsize=13)
ax.set_ylabel('Instrucciones ejecutadas', fontsize=13)
ax.set_title('Instrucciones vs. Tamano', fontsize=14, fontweight='bold')
ax.legend(fontsize=10)
ax.set_yscale('log'); ax.set_xscale('log', base=2)
ax.xaxis.set_major_formatter(ticker.ScalarFormatter())
ax.grid(True, which='both', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.savefig('grafica_instrucciones.pdf', dpi=150, bbox_inches='tight')
plt.savefig('grafica_instrucciones.png', dpi=150, bbox_inches='tight')

fig, ax = plt.subplots(figsize=(9, 6))
ax.plot(n_t, tb_t, 'o-', color='#e74c3c', lw=2, ms=7,
        label='Fuerza Bruta (experimental)')
ax.plot(n_t, ts_t, 's-', color='#2980b9', lw=2, ms=7,
        label='Strassen Hibrido (experimental)')
ax.plot(n_t, C_bruta_time * n_t**3, '--', color='#c0392b', lw=1.5,
        label=f'C1*n^3 (C1={C_bruta_time:.2e})')
ax.plot(n_t, C_strassen_time * n_t**exp_strassen, '--',
        color='#1a5276', lw=1.5,
        label=f'C2*n^log2(7) (C2={C_strassen_time:.2e})')
ax.set_xlabel('Tamano n', fontsize=13)
ax.set_ylabel('Tiempo (segundos)', fontsize=13)
ax.set_title('Tiempo vs. Tamano', fontsize=14, fontweight='bold')
ax.legend(fontsize=10)
ax.set_yscale('log'); ax.set_xscale('log', base=2)
ax.xaxis.set_major_formatter(ticker.ScalarFormatter())
ax.grid(True, which='both', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.savefig('grafica_tiempo.pdf', dpi=150, bbox_inches='tight')
plt.savefig('grafica_tiempo.png', dpi=150, bbox_inches='tight')

fig, ax = plt.subplots(figsize=(9, 5))
razon_instr  = i_bru / i_str
razon_tiempo = tb_t / ts_t
teorica_all  = n**0.193
ax.plot(n,   razon_instr,  'o-', color='#8e44ad', lw=2, ms=7,
        label='Razon instrucciones (Bruta/Strassen)')
ax.plot(n_t, razon_tiempo, 's-', color='#27ae60', lw=2, ms=7,
        label='Razon tiempo (Bruta/Strassen)')
ax.plot(n, teorica_all, '--', color='gray', lw=1.5,
        label='Curva teorica n^0.193')
ax.axhline(y=1, color='black', linestyle=':', lw=1.2,
```

```
            label='Empate (razon=1)')
ax.set_xlabel('Tamano n', fontsize=13)
ax.set_ylabel('Razon Bruta/Strassen', fontsize=13)
ax.set_title('Ventaja relativa de Strassen', fontsize=14, fontweight='
    bold')
ax.legend(fontsize=10)
ax.set_xscale('log', base=2)
ax.xaxis.set_major_formatter(ticker.ScalarFormatter())
ax.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.savefig('grafica_razon.pdf', dpi=150, bbox_inches='tight')
plt.savefig('grafica_razon.png', dpi=150, bbox_inches='tight')
plt.show()
```

Listing 4: Código completo de `graficar.py`

5. Resultados Experimentales

5.1. Metodología

Para cada tamaño $n \in \{2, 4, 8, 16, 32, 64, 128, 256, 512, 1024\}$ se generaron **3 instancias independientes** con matrices aleatorias distintas. El uso de tres repeticiones permite promediar la variabilidad del tiempo de ejecución causada por el sistema operativo [7]. Los valores en las columnas `_avg` son los **promedios** de las tres repeticiones.

Significado de las columnas del resumen:

- **n**: dimensión de la matriz ($n \times n$).
- **tiempo_strassen_avg**: tiempo promedio de Strassen (s).
- **instrucciones_strassen_avg**: instrucciones simples promedio de Strassen.
- **tiempo_bruta_avg**: tiempo promedio de fuerza bruta (s).
- **instrucciones_bruta_avg**: instrucciones simples promedio de fuerza bruta.

5.2. Fase 1 — Strassen Puro (sin umbral híbrido)

En la primera fase se implementó Strassen con caso base $n = 1$, sin umbral. Los resultados mostraron que Strassen era *más lento* que la fuerza bruta en todos los tamaños [3].

Cuadro 1: Resultados con Strassen puro (caso base $n = 1$, sin umbral)

n	T. Strassen (s)	Instruc. Strassen	T. Bruta (s)
64	0.317	1 390 436	0.003
128	2.324	9 855 933	0.023
256	16.31	69 483 052	0.251

5.3. Fase 2 — Strassen Híbrido (umbral $n_0 = 32$)

Con el umbral $n_0 = 32$, Strassen supera consistentemente a la fuerza bruta para $n \geq 128$ [4].

Cuadro 2: Strassen híbrido ($n_0 = 32$) — promedios de 3 repeticiones

n	T. Strassen (s)	Instruc. Strassen	T. Bruta (s)	Instruc. Bruta
2	0.000000	10	0.000000	26
4	0.000000	66	0.000000	164
8	0.000174	514	0.000000	1 160
16	0.000000	4 098	0.000000	8 720
32	0.000000	32 770	0.000333	67 616
64	0.003936	260 111	0.000000	532 544
128	0.019005	1 943 658	0.034967	4 227 200
256	0.201482	14 097 127	0.242066	33 685 760
512	1.547443	100 645 970	2.055370	268 960 256
1024	11.59157	712 386 111	27.16043	2 149 581 824

Observaciones clave:

- Para $n \leq 64$: la fuerza bruta es más rápida; el overhead de partición supera el ahorro.
- Para $n \geq 128$: **Strassen gana en tiempo y en instrucciones**.
- Para $n = 1024$: Strassen es **2.3 veces más rápido** y ejecuta **3 veces menos instrucciones**.
- La razón de instrucciones para $n = 1024$ es $2\,149\,581\,824/712\,386\,111 \approx 3.02$, consistente con $1024^{0,193} \approx 3.0$.

6. Análisis Gráfico

Las gráficas fueron generadas con `graficar.py` a partir de `resumen.csv`. Las constantes se calcularon promediando los cocientes para $n \geq 128$:

$$C_1 = \overline{\frac{\text{instrucciones_bruta}}{n^3}} = 2.0073, \qquad C_2 = \overline{\frac{\text{instrucciones_strassen}}{n^{\log_2 7}}} = 2.4554.$$

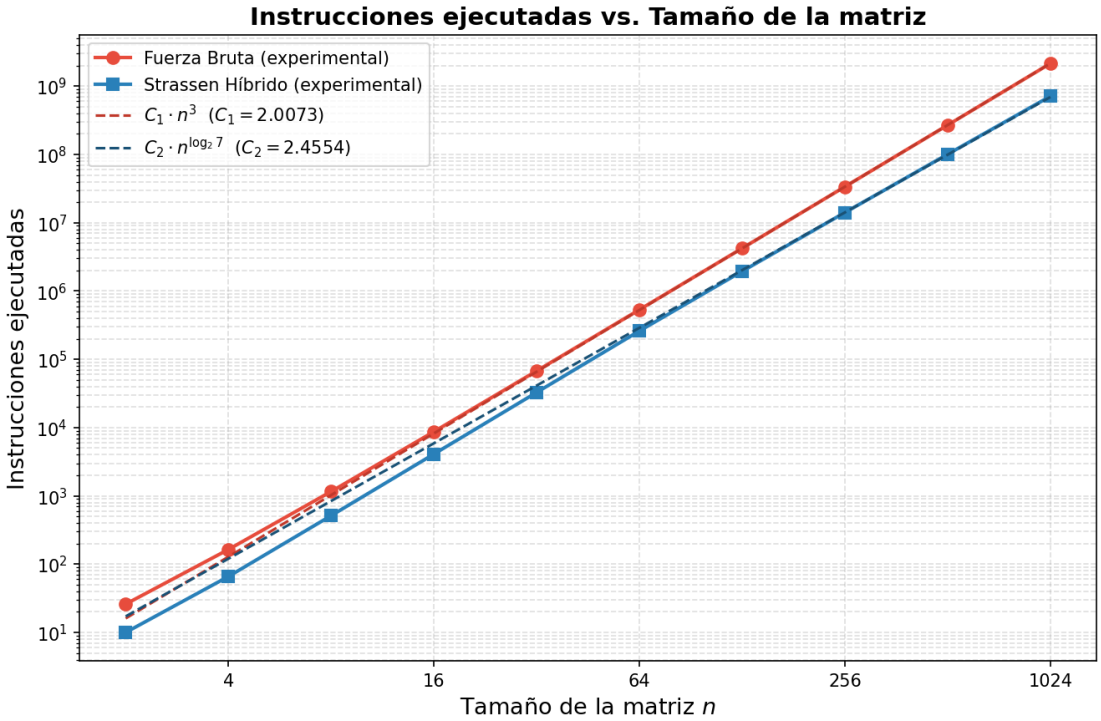

Figura 1: Número de instrucciones ejecutadas vs. tamaño n (escala log-log). Líneas sólidas: datos experimentales. Líneas discontinuas: curvas teóricas $C_1 \cdot n^3$ ($C_1 = 2.0073$) y $C_2 \cdot n^{\log_2 7}$ ($C_2 = 2.4554$).

La Figura 1 muestra que las curvas teóricas se ajustan con gran precisión a los datos experimentales. La brecha entre rojo (fuerza bruta) y azul (Strassen) crece visiblemente con n, confirmando que $O(n^{2.807})$ escala mejor que $O(n^3)$.

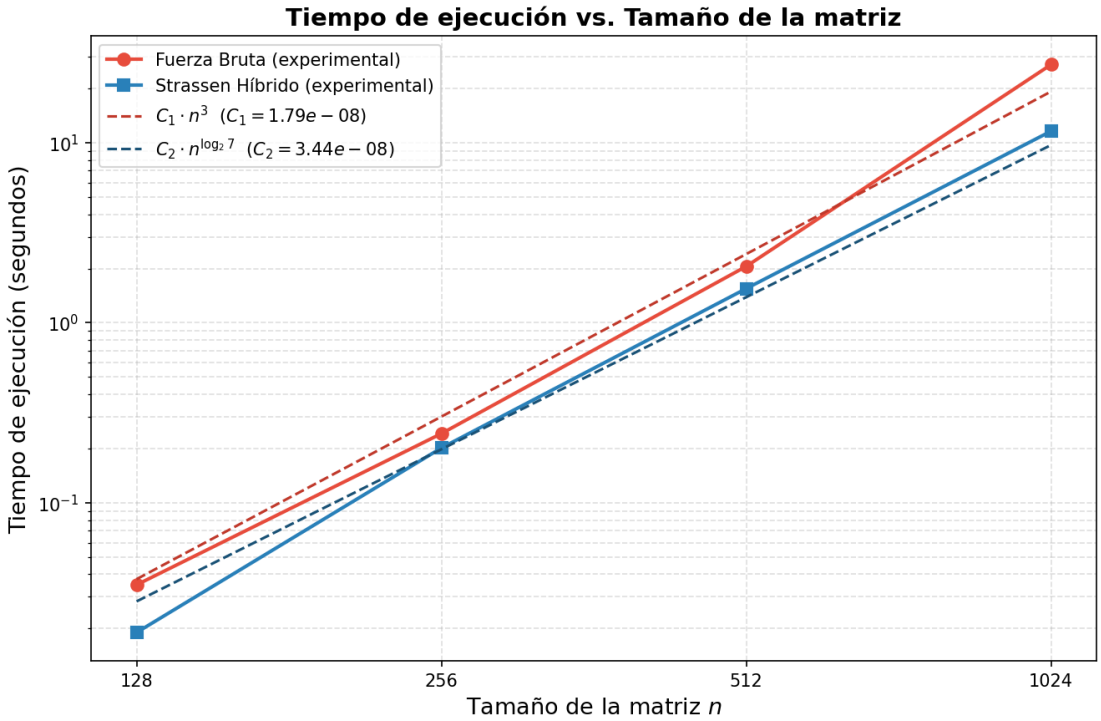

Figura 2: Tiempo de ejecución vs. tamaño n (escala log-log, solo $n \geq 128$). Constantes: $C_1 = 1.79 \times 10^{-8}$ s y $C_2 = 3.44 \times 10^{-8}$ s.

La Figura 2 confirma que para $n \geq 128$ Strassen es más rápido. El hecho de que $C_2 > C_1$ refleja el mayor costo por instrucción de Strassen debido al overhead de partición [3].

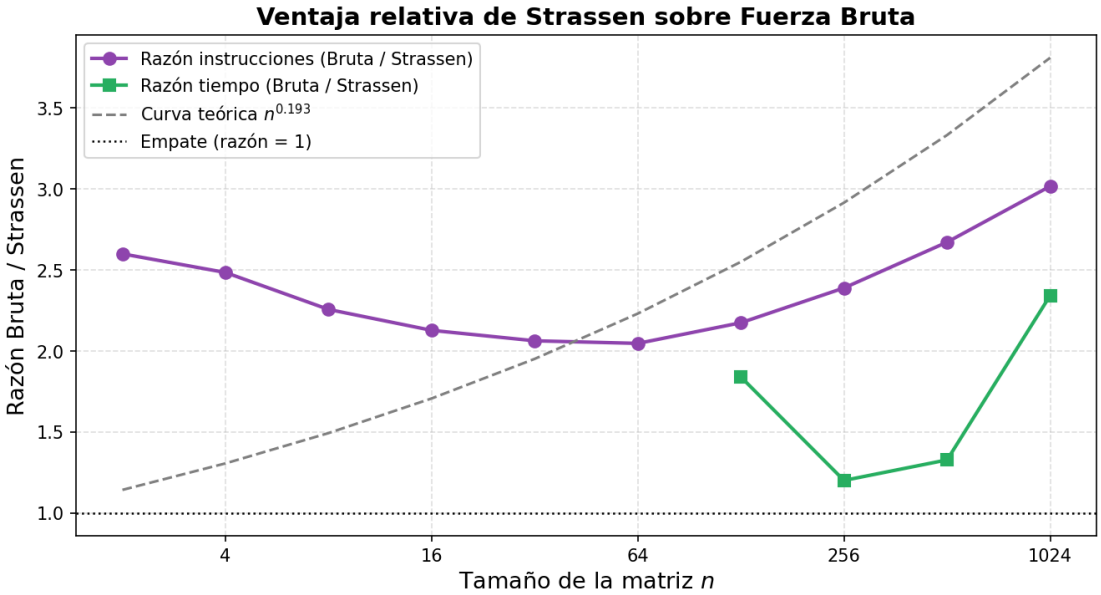

Figura 3: Razón Bruta/Strassen de instrucciones (morado) y tiempo (verde). Valores > 1 indican ventaja de Strassen. Curva gris: tendencia teórica $n^{0.193}$. Línea punteada: empate (razón $= 1$).

La Figura 3 muestra que la razón de instrucciones siempre supera 2 y crece hacia 3 en

$n = 1024$. La razón de tiempo alcanza ≈ 2.3 en $n = 1024$.

7. Conclusiones

A partir de la implementación, instrumentación y análisis de los algoritmos, se derivan las siguientes conclusiones:

1. **Verificación empírica de la complejidad asintótica**: Al duplicar n, la fuerza bruta multiplica su trabajo por ≈ 8 (consistente con $O(n^3)$) y Strassen por ≈ 7 (consistente con $O(n^{\log_2 7})$). Esto demuestra que la reducción de subproblemas en Dividir y Conquistar ofrece una ventaja matemática insuperable para entradas grandes [1].

2. **El impacto de las constantes ocultas**: Para $n \leq 64$, Strassen requiere más instrucciones y tiempo que la fuerza bruta. Esto comprueba que la constante C en $T(n) = C \cdot f(n)$ es significativamente mayor en Strassen, debido al overhead de 18 sumas/restas de matrices, la reserva de memoria dinámica y las llamadas recursivas [3].

3. **Identificación del punto de cruce (*crossover point*)**: La tabulación de tiempos permitió identificar empíricamente que el cruce ocurre a partir de $n = 128$, ampliando la ventaja al llegar a $n = 512$ y $n = 1024$ [4].

4. **Efectividad de la arquitectura híbrida**: El Strassen con umbral híbrido $n_0 = 32$ mitiga la ineficiencia en matrices pequeñas preservando la ventaja teórica en problemas masivos, reflejando los estándares de las librerías matemáticas de la industria [4, 6].

5. **Complejidad teórica vs. práctica**: Sin el umbral híbrido, Strassen no supera a la fuerza bruta en los rangos probados, ilustrando que la complejidad asintótica por sí sola no determina el rendimiento real [7].

6. **Dividir y Conquistar como paradigma de diseño**: Strassen ilustra cómo dividir un problema en subproblemas más pequeños produce una mejora asintótica real, formalizada por el Teorema Maestro, aplicable a una gran variedad de problemas computacionales [1, 7].

Referencias

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[2] V. Strassen, "Gaussian elimination is not optimal," *Numerische Mathematik*, vol. 13, no. 4, pp. 354–356, 1969.

[3] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA: SIAM, 2002.

[4] A. R. Benson and G. Ballard, "A framework for practical parallel fast matrix multiplication," in *Proc. ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP)*, Orlando, FL, Feb. 2014, pp. 42–53.

[5] V. Y. Pan, "How can we speed up matrix multiplication?" *SIAM Review*, vol. 26, no. 3, pp. 393–415, Jul. 1984.

[6] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD: Johns Hopkins University Press, 2013.

[7] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Cham, Switzerland: Springer, 2020.